Ghulam Ishaq Khan Institute of Engineering Sciences and Technology
Faculty of Computer Science and Engineering
CY321 – Secure Software Development

Final Project Report

Enigma: Zero-Trust Secure Vault and Active Defence Web Application

---

Developer: Aayan Rashid
Student Reg: 2023002
Course: CY321 – Secure Software Development
Institution: Ghulam Ishaq Khan Institute of Engineering Sciences and Technology (GIKI)
Submission: Final Project Deliverable

1. Project Overview

Enigma is a security-focused web application for the one-time, end-to-end encrypted sharing of sensitive information such as passwords, credentials, and confidential messages. The system is built on a Zero-Trust architecture in which the server is treated as an untrusted intermediary: encryption keys are generated exclusively in the browser and are never transmitted to or stored on the backend. This guarantees that a fully compromised server cannot expose plaintext secrets.

The application provides the following capabilities:

- **Client-side encryption:** AES-256-GCM is performed entirely in the browser via the WebCrypto API; no plaintext crosses the network.
- **Burn-After-Reading:** Each encrypted payload is deleted from the database upon first successful retrieval, enforcing single-access semantics.
- **Active defence via HoneyLinks:** Decoy links with synthetic identifiers are generated and distributed. Any access triggers structured logging of the intruder's IP and User-Agent without revealing the trap.
- **Structured payload encryption:** Both Password objects (URL, username, password) and Secure Message objects are serialised, validated, and encrypted before transmission.
- **Hardened API security:** The backend uses Helmet.js for HTTP headers, express-rate-limit for DoS mitigation, strict CORS policy, and bounded JSON payload parsing.
- **Defensive coding:** Each implementation decision maps directly to a CY321 learning outcome, as detailed in Section 6.

2. Final Technology Stack

Layer	Technology / Component
Frontend	HTML5, CSS3, Vanilla JavaScript (no framework dependencies)
Cryptography	WebCrypto API — AES-256-GCM with per-note randomly generated 12-byte IV
Backend	Node.js with Express.js web framework
Database	Supabase-managed PostgreSQL via @supabase/supabase-js client library
Security Middleware	helmet, express-rate-limit, cors, JSON body parser (10 KB limit)

3. Functional Architecture

3.1 Client-Side Layer (Browser)

The interface exposes two input modes: a Password tab (URL, username, password) and a Secure Message tab (free-form text). On submission the following processing pipeline executes in the browser:

- User inputs are serialised to a JSON string and validated against a 5,000-byte ceiling.
- A 256-bit AES-GCM key is generated via window.crypto.subtle.generateKey.
- The JSON payload is encrypted in-browser using the generated key and a randomly generated 12-byte IV.
- Only the opaque ciphertext and IV are sent to the backend via POST /api/notes.
- The raw key is exported, Base64-encoded, and appended to the shareable URL as a hash fragment (e.g. /view/<id>#<KEY>). Because browsers do not include hash fragments in HTTP requests, the key is never received by the server.

3.2 Server-Side Layer (Express + Supabase)

The Express backend exposes two API endpoints:

- **POST /api/notes:** Persists the encrypted payload (id, ciphertext, iv) to the Supabase notes table. No plaintext or key material is accepted or stored.
- **GET /api/notes/:id:** Implements Burn-After-Reading and active defence logic:
  - If the ID matches the honey_ prefix, the server logs the source IP and User-Agent to intruder_logs and returns a generic 404. No indication is given that a trap was triggered.
  - For a legitimate ID, the record is fetched and immediately deleted within the same operation. The ciphertext and IV are returned for in-browser decryption.

4. Security Controls Implemented

4.1 Zero-Trust Key Handling

The AES decryption key is transported exclusively via the URL hash fragment. Per the HTTP specification and all major browser implementations, hash fragments are client-side constructs and are never included in HTTP requests. The backend therefore stores no key material. Even a fully compromised database yields only opaque ciphertext with no viable decryption path.

4.2 Burn-After-Reading Semantics

On the first successful GET for a given note identifier, the record is fetched and deleted from Supabase before the response is returned. The encrypted payload is available exactly once. All subsequent requests for the same ID return 404, protecting against replay attacks by observers who intercept the link after its intended use.

4.3 Active Defence — HoneyLinks

HoneyLinks use a honey_ prefix followed by a random hex string. When accessed, the server captures the request IP and User-Agent and persists them to intruder_logs, then returns a standard 404. The adversary receives no feedback indicating their activity was detected or logged.

4.4 Input Validation and Defensive Coding

- **Frontend validation:** The serialised payload is checked against a 5,000-byte hard limit before encryption. Oversized inputs are rejected before any cryptographic or network operation.
- **Backend validation:** The Express JSON body parser is capped at 10 KB, preventing large-body denial-of-service vectors at the HTTP layer.
- **Error handling:** All responses are sanitised. Stack traces and database diagnostics are logged server-side only and never forwarded to the client.

4.5 XSS Mitigation

Decrypted content is rendered exclusively through textContent assignment and structured DOM creation (createElement, appendChild). The innerHTML property is never used for any user-controlled or decrypted output, eliminating the primary DOM-based XSS attack surface.

4.6 Memory Safety and Secret Lifecycle Management

After each cryptographic operation, all sensitive references — including the raw payload string, the CryptoKey object, the IV buffer, and the ciphertext buffer — are explicitly nulled. JavaScript's garbage collector does not guarantee immediate reclamation, but this releases the references from the live object graph at the earliest opportunity, narrowing the window for a memory inspection attack.

4.7 Plausible Deniability on Decryption Failure

If decryption fails — due to an incorrect key, tampered ciphertext, or corrupted IV — the application does not throw an exception, display a cryptographic error, or give any indication a decryption attempt occurred. A contextually plausible operational message is rendered instead, preventing an adversary from using error responses as a decryption oracle.

4.8 Web Security Header Hardening

- **Helmet.js:** Enforces Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, and Strict-Transport-Security at the Express application level.
- **Rate limiting:** express-rate-limit throttles requests per source IP across all /api routes, reducing brute-force and DoS exposure.
- **CORS policy:** Cross-origin requests are restricted to explicitly authorised origins.

5. Database Design (Supabase PostgreSQL)

5.1 notes Table

Column	Type	Description	Constraint
id	text	Primary identifier (UUID or custom string)	NOT NULL, PRIMARY KEY
ciphertext	text	AES-GCM encrypted payload, Base64-encoded	NOT NULL
iv	text	Initialisation vector, Base64-encoded	NOT NULL
created_at	timestamptz	Record creation timestamp	DEFAULT now()

5.2 intruder_logs Table

Column	Type	Description	Constraint
id	uuid	Auto-generated record identifier	DEFAULT gen_random_uuid()
ip	text	Source IP address of the intruding request	NOT NULL
user_agent	text	HTTP User-Agent string captured from request	NULLABLE
created_at	timestamptz	Capture timestamp	DEFAULT now()

6. CY321 Syllabus Outcome Mapping

The table below maps each implemented security control directly to its corresponding CY321 course topic and learning outcome.

Course Topic	Enigma Implementation	Outcome Achieved
Weeks 2 & 4 Secure SDLC + STRIDE	HoneyLink-based active defence: trap ID pattern detection and structured intruder telemetry logging	Reconnaissance and Spoofing threat categories explicitly mitigated through deceptive response infrastructure
Weeks 7 & 8 Defensive Coding	Client-side 5,000-byte payload ceiling; XSS prevention via textContent rendering exclusively	Input validation and injection prevention enforced at both the frontend and API layers

Course Topic	Enigma Implementation	Outcome Achieved
Week 9 Secure Implementation	Explicit nullification of key, iv, cipherBuffer, and payloadString after cryptographic operations	Sensitive reference lifetime in runtime memory reduced to the minimum necessary window
Week 12 Brute-Force Mitigation	Plausible deniability on decryption failure; no deterministic cryptographic error disclosure	Decryption oracle attack surface eliminated; failure mode indistinguishable from normal operation
Weeks 13 & 14 Web Security Defences	Helmet.js headers; API rate limiting; payload cap; Zero-Trust hash-fragment key transport; Burn-After-Reading	Full web-layer hardening aligned with OWASP Top 10 and course-specified defence patterns

7. Threat Model

7.1 Protected Assets

- Plaintext secret payloads (passwords, credentials, confidential messages) prior to encryption.
- AES-256-GCM encryption keys, which exist only in the client browser and the URL hash fragment.
- Shareable one-time access links, which are the sole access credential for a given note.

7.2 Threat Actors and Mitigations

- **Passive network observer:** Monitors traffic or inspects TLS-decrypted payloads. Mitigated by client-side encryption; only ciphertext transits the network.
- **Compromised server or database:** Attacker has direct read access to Supabase. Mitigated by Zero-Trust key architecture; the database holds only ciphertext with no key material.
- **Link guesser / brute-force scanner:** Iterates over possible note identifiers. Mitigated by rate limiting, UUID-based identifier space, and Burn-After-Reading deletion.
- **Opportunistic scanner:** Probes for exposed endpoints. Mitigated by HoneyLink telemetry, which silently captures and logs scanner activity.
- **Replay attacker:** Intercepts a valid link after legitimate use. Mitigated by server-side deletion on first retrieval.

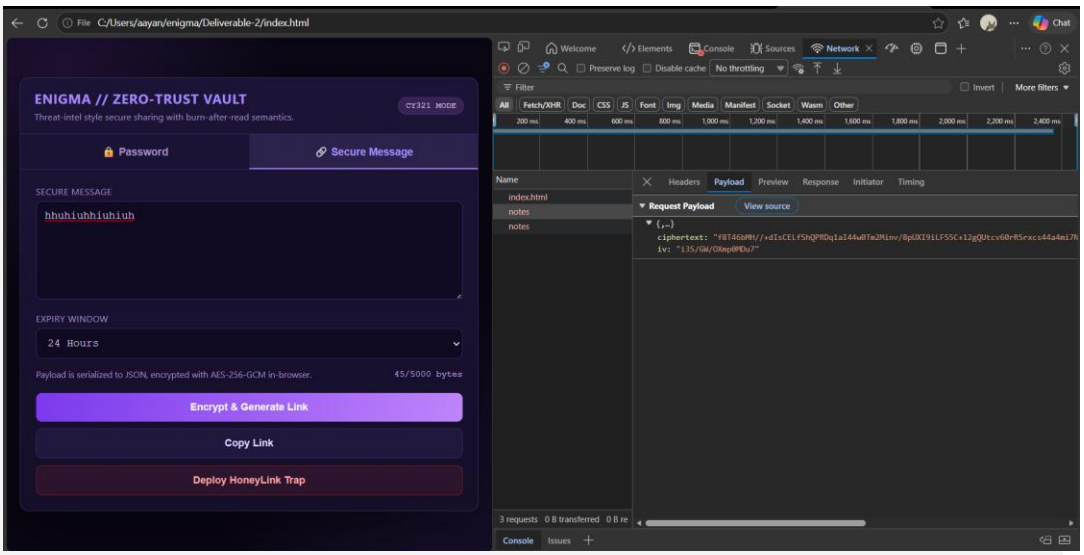
8. Testing and Validation (Deliverable 3 Evidence)

8.1 Test Case 1 — Zero-Trust Key Transmission

Objective: Confirm the AES decryption key is never transmitted to the backend during note creation or any subsequent API interaction.

Procedure: Browser developer tools (Network tab) were used to inspect raw HTTP request bodies and headers during note creation. The URL fragment content was confirmed to be absent from all captured traffic.

Expected result: The POST /api/notes request body contains only ciphertext and iv. No key material or hash fragment appears in any captured request.



Network inspector showing POST body containing only ciphertext and IV with no key material

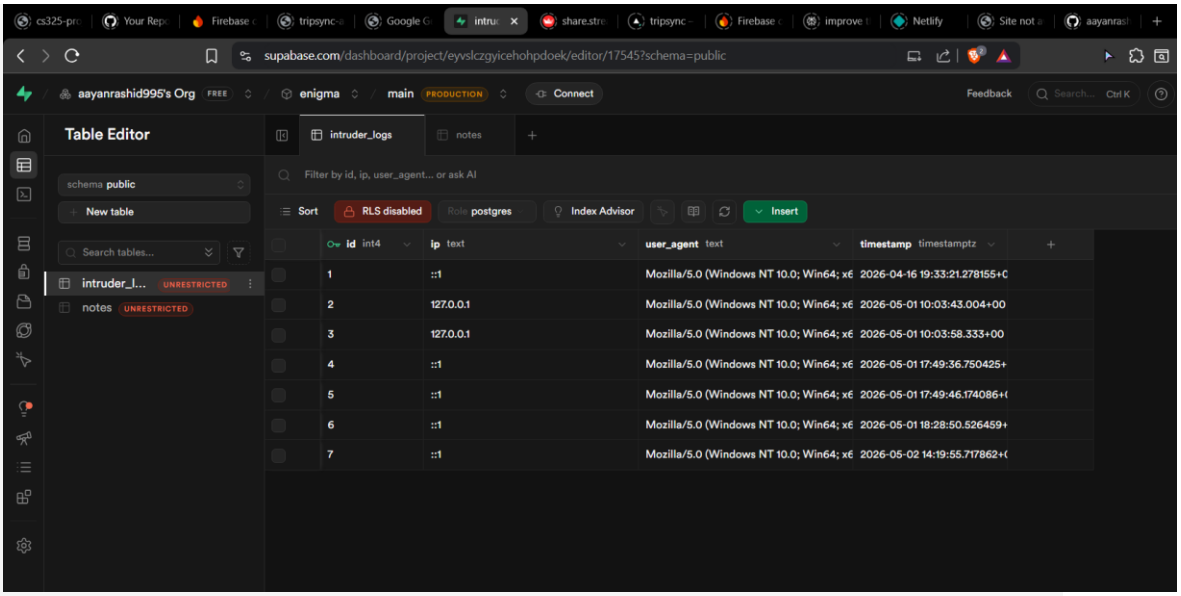
Figure 1: Zero-Trust Verification — POST /api/notes Request Payload

8.2 Test Case 2 — Active Defence Telemetry

Objective: Confirm a generated HoneyLink successfully captures intruder metadata and persists it to the database.

Procedure: A HoneyLink was generated and accessed from a controlled test environment. The Supabase intruder_logs table was inspected to verify a new record was created containing the correct source IP and User-Agent.

Expected result: A row is inserted into intruder_logs with the correct IP and User-Agent. The HoneyLink access returns a standard 404 with no indication of the logging activity.



Supabase dashboard showing captured intruder record with IP and User-Agent populated

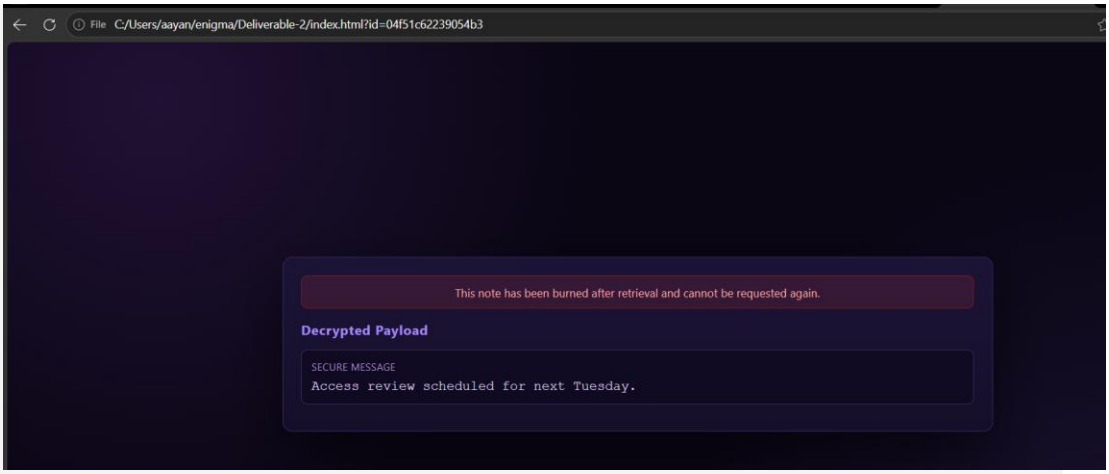
Figure 2: Active Defence Telemetry — Supabase intruder_logs Table

8.3 Test Case 3 — Plausible Deniability / Fail-Secure

Objective: Confirm that an incorrect decryption key or tampered ciphertext triggers the plausible deniability response rather than an error disclosure.

Procedure: A valid note was created and the shareable link was modified by altering the hash fragment key. The link was opened and the resulting application state was observed.

Expected result: A contextually plausible operational message is rendered. No JavaScript exception is thrown, no error modal appears, and no information about the failed decryption is exposed.



Application UI showing decoy message rendered after an invalid key was supplied

Figure 3: Plausible Deniability — Decoy Response on Decryption Failure

9. Known Limitations and Future Work

9.1 Current Limitations

- **No user authentication:** Intentionally omitted to remain within course deliverable scope. A production deployment would require an authentication layer to associate notes with verified senders and restrict access to intruder log dashboards.
- **JavaScript memory nullification constraints:** Explicit nullification is a best-effort measure. JavaScript's garbage-collected runtime does not provide deterministic memory reclamation equivalent to explicit wiping in C or C++.
- **No note expiry mechanism:** Unretrieved notes persist indefinitely. A TTL mechanism or scheduled cleanup job would be required for production readiness.

9.2 Proposed Enhancements

- **Passphrase-derived key wrapping:** An optional PBKDF2 or Argon2 layer would allow notes to be protected by a shared passphrase in addition to the URL-fragment key.
- **Audit dashboard for intruder logs:** A protected administrative interface providing visualisation and alerting for intruder_logs events.
- **Note expiry and TTL enforcement:** Automatic deletion of unretrieved notes after a configurable period.

10. Conclusion

Enigma demonstrates a practical, secure-by-design implementation of one-time secret sharing under a Zero-Trust model. The project integrates client-side AES-256-GCM cryptography, Zero-Trust key transport via URL hash fragments, Burn-After-Reading deletion, and structured active defence telemetry into a single cohesive application.

Each security control maps to a CY321 learning outcome, covering the secure SDLC, defensive coding, secure implementation, cryptographic protocol design, and web-layer security defence. The core architectural decision — treating the server as untrusted — ensures that confidentiality guarantees hold even under a worst-case infrastructure compromise scenario.

Signature: _____ Date: _____

Aayan Rashid | 2023002 | CY321 — Secure Software Development